

Lab9 Loop Invariant Code Motion

1. main.cc

对每个函数先调用 `findLoopHeaders`，识别这个函数里有哪些 loop header，以及每个 loop 包含哪些基本块。把识别到的 loop 信息打印到标准输出。

再调用 `loopHoistFunc`，在已经知道 loop 结构的前提下做 loop invariant code motion，也就是把循环里每次都算同样结果的语句提到 preheader。最后把优化后的所有函数重新拼成一个程序。

2. findloopheader.cc

核心函数：`findLoopHeaders(func, ffi)`，根据函数的控制流信息分析当前函数的循环。

枚举一条边 `source -> target`，查 `source` 的 dominators。如果 `target` 在 `source` 的 dominator 集合里，那这条边就是回边，`target` 就是 loop header。因为支配关系表示：从函数入口走到 `source` 的任何路径，都必须先经过 `target`，此时又有 `source -> target` 的边，自然构成循环。然后沿着 `source` 的前驱回溯，把经过所有节点都加入到循环体内，即可得到完整的循环体。

最后将循环信息封装为 `loopHeaders`。

3. loophoistfunc.cc

3.1 loopHoistFunc(func, loopHeaderMap)

使用 `findloopheader` 得到的 `loopHeaderMap`，取出所有 loop，进行排序：bodyBlocks 少的排前面，如果大小一样，headerLabel 小的排前面。因为对于多层循环，内层 loop 中可能有表达式被提到 preheader，再在外层中再次被提出，这样可以先处理内层循环再处理外层循环。

通过 `buildPredecessors(func)` 找 header 的所有前驱，找到其中位于循环外部的那个前驱块，作为 preheader。

迭代发现循环不变量：每一轮里重新 `buildDefBlockMap(func)`，然后遍历当前 `loopBlocks` 中的每个语句，检查。如果语句类型允许 hoist，且 `def` 只有一个 `temp`，`use` 的变量都属于循环外或者已知的 invariant temp，那么将这个语句认定为循环不变量，进行提升。

提升过程：调用 `insertIntoPreheader(preheader, stmt)` 插入到 preheader，从原 block 中删除，并把定义的 `temp` 加入 `invariantTemps`。

允许 hoist 的语句：MOVE/MOVE_BINOP/PTR_CALC。其余语句可能存在副作用，每个 memory load 都可能获取到不同的值。

3.2 buildDefBlockMap

遍历所有 block 里的 stmt，看 `stmt->def` 里定义了哪些 temp，记录这些 temp 的定义块是当前 blockLabel。供 `loopHoistFunc` 在循环迭代时检查 `use` 的变量在何处被定义。

3.3 insertIntoPreheader

先算一个 insertPos: 默认是 block 尾部, 但如果块尾是一条 terminator, 那就把插入位置前移一格。
将要 hoist 的表达式插入到正确位置。

4. git graph

